```
import os
import re
import json
import requests
from supabase import create_client
from datetime import datetime, timedelta

sb = create_client(os.environ["SUPABASE_URL"], os.environ["SUPABASE_KEY"])

ZIP_CODE = "79639"

STORE_MAP = {
    "lidl": "lidl_de",
    "aldi": "aldi_de",
    "rewe": "rewe_de",
    "edeka": "edeka_de",
    "kaufland": "kaufland_de",
    "penny": "penny_de",
}

PRODUCTS = [
    "Milch","Eier","Butter","Kaese","Joghurt",
    "Haehnchen","Hackfleisch","Lachs","Wurst","Schinken",
    "Brot","Nudeln","Reis","Mehl","Zucker","Oel",
    "Apfel","Banane","Tomate","Karotte","Kartoffel","Paprika",
    "Wasser","Saft","Kaffee","Bier","Wein",
    "Chips","Schokolade","Kekse"
]

HEADERS = {"User-Agent": "Mozilla/5.0 (iPhone; CPU iPhone OS 17_0 like Mac OS X)"

def get_api_key():
    try:
        resp = requests.get("https://www.marktguru.de/", timeout=10, headers=HEAD
        for pattern in ["\"api_key\"\\s*:\\s*\"([^\"]+)\"", "\"apiKey\"\\s*:\\s*\
            m = re.search(pattern, resp.text, re.IGNORECASE)
            if m:
                print("Found API key")
                return m.group(1)
    except Exception as e:
        print("Key error: " + str(e))
    return "mg-api-key"

def search_offers(query, api_key):
```

```
    try:
        url = "https://api.marktguru.de/api/v1/offers/search"
        headers = {
            "x-apikey": api_key,
            "accept": "application/json",
            "User-Agent": "Mozilla/5.0"
        }
        params = {
            "as": "web",
            "limit": 15,
            "offset": 0,
            "q": query,
            "zipCode": ZIP_CODE
        }
        resp = requests.get(url, headers=headers, params=params, timeout=10)
        if resp.status_code == 200:
            data = resp.json()
            return data.get("offers", data.get("results", []))
        else:
            print("Status " + str(resp.status_code) + " for " + query)
    except Exception as e:
        print("Error " + query + ": " + str(e))
    return []

def parse_rows(offers):
    rows = []
    seen = set()
    week_end = (datetime.utcnow() + timedelta(days=7)).strftime("%Y-%m-%d")

    for o in offers:
        try:
            name = o.get("description", o.get("product", {}).get("name", ""))
            price = float(o.get("price", 0) or 0)
            old_price = float(o.get("oldPrice") or o.get("referencePrice") or pri
            offer_id = str(o.get("id", ""))
            image_url = "https://images.marktguru.de/offers/" + offer_id + "/w300

            advertisers = o.get("advertisers", [])
            if not advertisers:
                continue
            store_name = advertisers[0].get("name", "")

            unit_obj = o.get("unit", {})
            unit = unit_obj.get("shortName", "pcs") if isinstance(unit_obj, dict)

            validity = o.get("validityDates", [])
            valid_until = week_end
```

```
            if validity:
                valid_until = validity[0].get("to", week_end)[:10]

            if not name or not price or not store_name:
                continue

            store_id = None
            for key, sid in STORE_MAP.items():
                if key.lower() in store_name.lower():
                    store_id = sid
                    break
            if not store_id:
                continue

            pid = re.sub("[^a-z0-9]+", "_", name.lower()).strip("_")[:50]
            dedup = pid + "_" + store_id
            if dedup in seen:
                continue
            seen.add(dedup)

            rows.append({
                "store_id": store_id,
                "store_name": store_name,
                "store_flag": "DE",
                "product_id": pid,
                "product_name": name,
                "price": price,
                "old_price": old_price,
                "offer_id": offer_id,
                "image_url": image_url,
                "currency": "EUR",
                "unit": unit,
                "valid_until": valid_until,
                "created_at": datetime.utcnow().isoformat()
            })
        except Exception as e:
            print("Parse error: " + str(e))

    return rows

def main():
    api_key = get_api_key()
    print("Using key: " + api_key[:10] + "...")

    all_offers = []
    for product in PRODUCTS:
        offers = search_offers(product, api_key)
```

```
            all_offers.extend(offers)
            print(product + ": " + str(len(offers)) + " offers")

        print("Total raw: " + str(len(all_offers)))
        rows = parse_rows(all_offers)
        print("Parsed: " + str(len(rows)))

        if rows:
            for sid in set(r["store_id"] for r in rows):
                sb.table("prices").delete().eq("store_id", sid).execute()
            sb.table("prices").insert(rows).execute()
            print("Saved " + str(len(rows)) + " to Supabase!")
        else:
            print("Nothing to save")

if __name__ == "__main__":
    main()
```